OAuth

Kameswari Chebrolu

Department of CSE, IIT Bombay

https://miro.medium.com/v2/resize:fit:800/0*D57AWE-MKMCswQXW

Outline

- **OAuth background and terminology**
- Authorization Code Grant
- Implicit Grant
- OpenID Connect and SSO
- OAuth Vulnerabilities
- Defense Mechanisms
- Real Life Examples

Scenario

Oauth Service Provider
(Safeguards user data: Name, profile picture, email, contacts, calendar, photos, drive etc…)

Third Party Service;
Wants name, email, calendar, contacts of user

User
(does not want to create another login at Zoom and share all that info; nor reveal Google password credentials to Zoom)

- User should have control over what data to reveal to Zoom
- Google should not reveal specific data to anyone else other than to Zoom
- How to achieve?

Open Authorization (Oauth)

- A protocol that allows third-party services to access user's resources without exposing user's credentials (e.g. username and password) to third parties
  - An **authorization** framework
  - Users can decide which data they wish to share
    - Instead of full control of their account to a third party!

High Level Idea

- User contacts Zoom, which asks it to do Google sign-in
- User then contacts Google and authenticates; also confirms what data it wishes to reveal and to whom?
  - Google validates the user and creates a token and passes it to user
  - Token is associated with “who to reveal:Zoom” and “what to reveal: name, email, contact etc”
- User takes this token and passes it to Zoom
- Zoom contacts Google with the token
  - Google verifies token and passes data after confirming its indeed Zoom
    - Zoom needs to prove to Google it is Zoom and not someone else

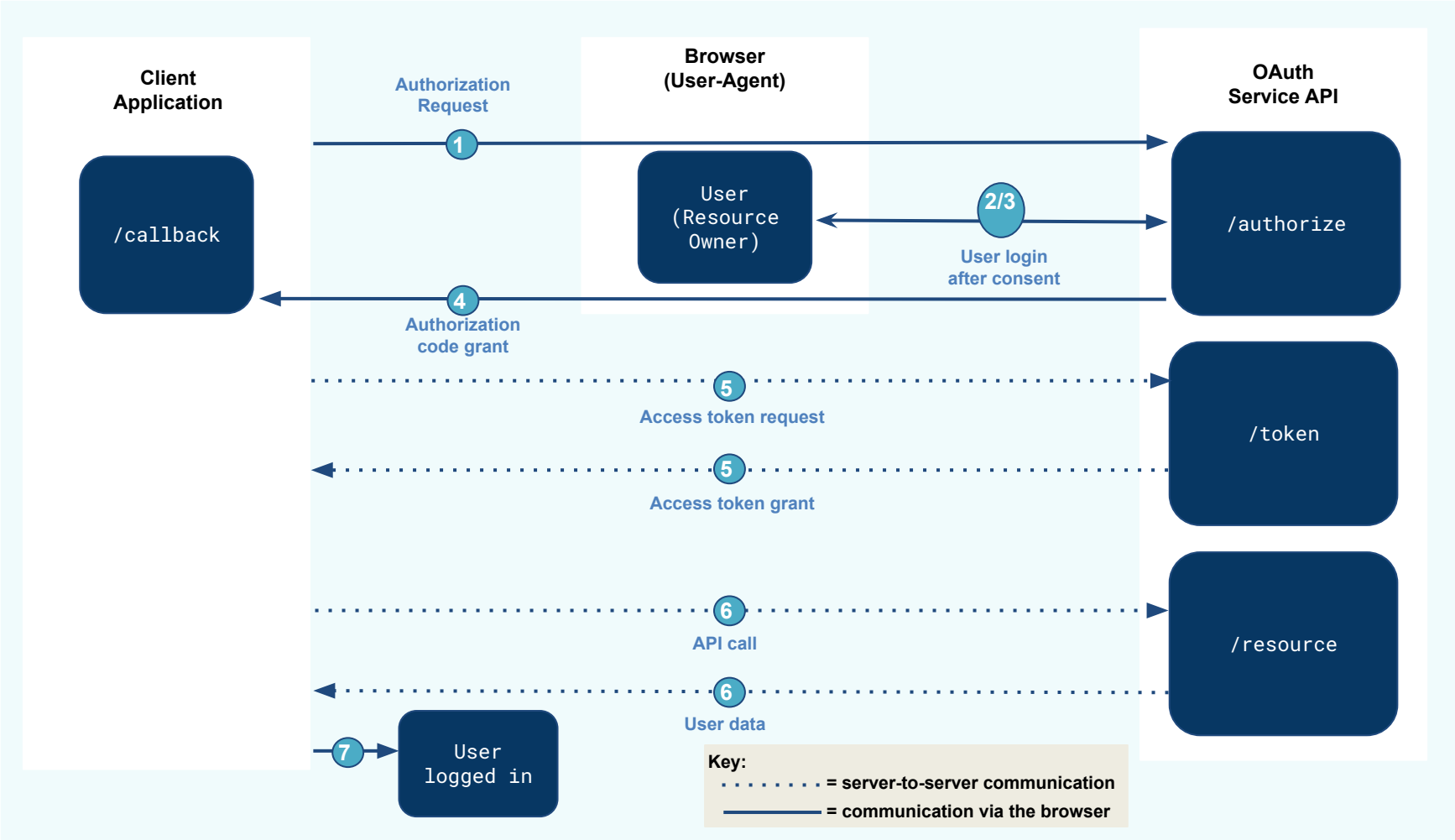
- OAuth versions:
  - OAuth 1.0: Published in 2010, complex and not very secure
  - OAuth 2.0: Introduced in 2012, a significant redesign; much simpler, more secure
    - Focuses on providing authorization while delegating authentication to other protocols like OpenID Connect
  - OAuth 2.1: Introduced in 2020. Set of best practices and recommendations for using OAuth 2.0 securely
- Our focus: OAuth 2.0

Oauth Flows

- Many different ways OAuth process can be implemented
  - Why? Caters to different use cases and security requirements
  - These are known as OAuth "flows" or "grant types"

Entities

- Client application: Website that wants access to user's data (e.g. Zoom)
- User: Resource owner (you)
- Oauth Service Provider: Application that controls user's data and has access to it (e.g. Google)
  - Often uses various API endpoints to achieve goal
    - /authorization: handles user authentication and consent
    - /token: issues access tokens, and verifies identity of client applications
    - /resource: hosts and protects user's resources and enforces access control based on supplied access tokens
    - Many times all of above implemented by same entity


Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

OAuth Flows

- Authorization Code Grant: Most common OAuth Flow
  - Best suited for server-side web applications
  - E.g. Website logins based on social media accounts
- Implicit Grant:
  - Suitable for client-side applications running in a web browser or mobile app
  - E.g. Single-page web application (SPA) or a mobile app accessing user data directly without a server intermediary
- We will focus on these two only!

- Client Credentials Grant:
  - Ideal for machine-to-machine authentication, does not require user involvement
  - E.g. Server-to-server communication or backend services accessing protected resources
- Resource Owner Password Credentials Grant:
  - Not recommended for most scenarios due to security risks
  - Legacy systems or first-party applications

- Refresh Token Grant: Complementary to other OAuth flows
  - Allows client-app to obtain new access tokens without requiring user re-authentication
  - Long-lived sessions or applications requiring continuous access to user data without frequent user interaction

OAuth Scopes

- In any grant, client application has to specify which data it wants to access and what operations it wants to perform
- Specified via scope parameter sent to OAuth Service Provider
  - An arbitrary text string → format can vary from provider to provider!
  - E.g. scope=contacts or scope=https://oauth-service-provider.com/auth/scopes/user/contacts.readonly

- When OAuth is used for authentication, standardized OpenID Connect scopes are often used
  - Ensure compatibility across providers!
  - E.g. scope=openid profile
    - Read access to a predefined set of basic information about user (e.g. email, userid etc)

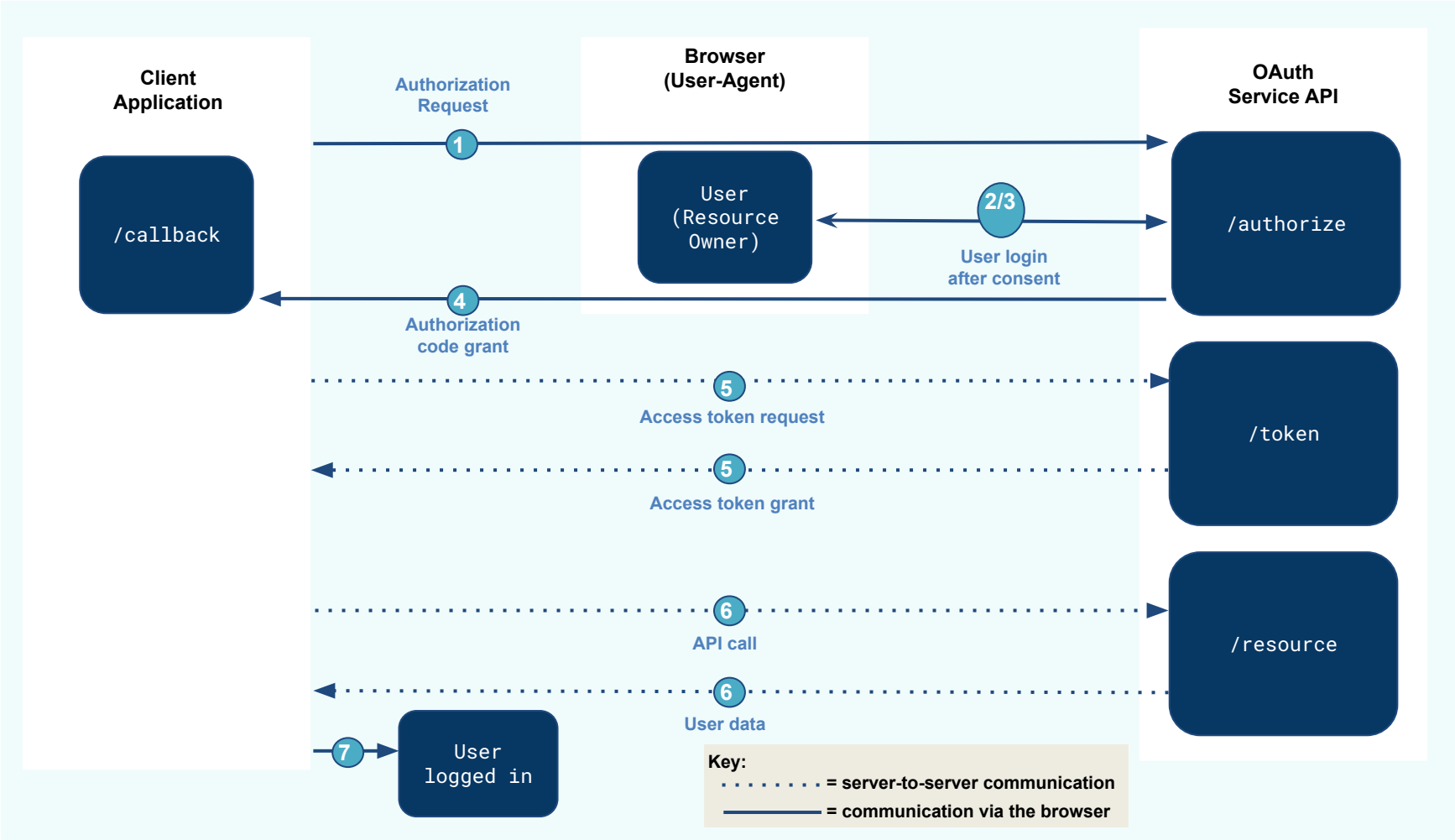
(More on OpenID Connect later)

Outline

- OAuth background and terminology
- **Authorization Code Grant**
- Implicit Grant
- OpenID Connect and SSO
- OAuth Vulnerabilities
- Defense Mechanisms
- Real Life Examples

Authorization Code Grant

Explain through diagrams


Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

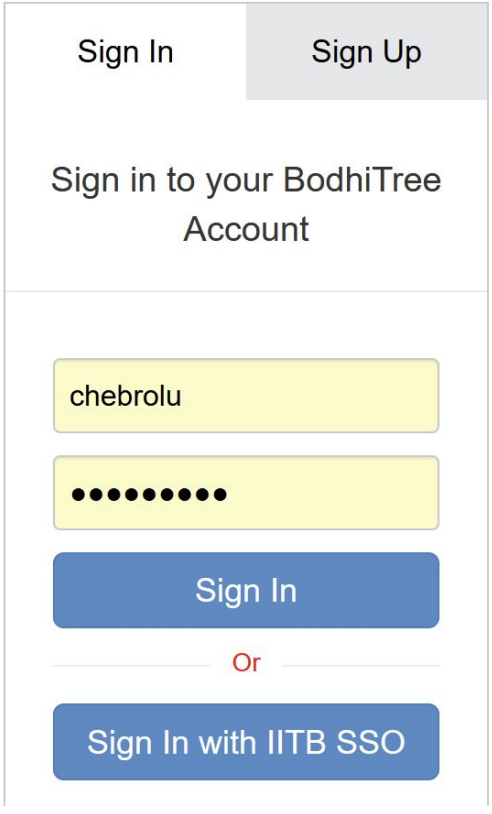
Implementations/Message-Sequence will differ in practice; but below captures the spirit!

1. User clicks on IITB SSO

GET /accounts/sso/ HTTP/2
Host: flamingo.bodhi.cse.iitb.ac.in

HTTP/2 302
…..

location=https://sso.iitb.ac.in/authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv

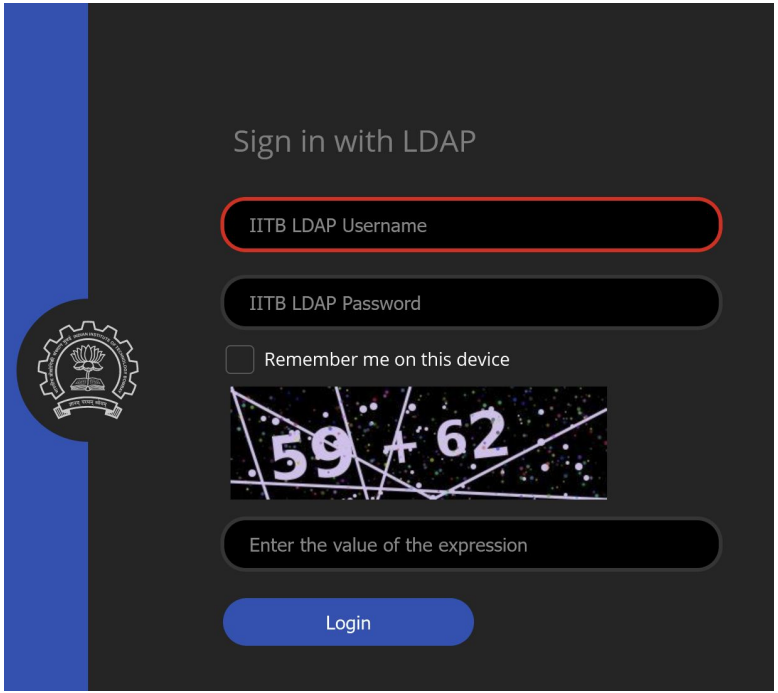


2.

GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv

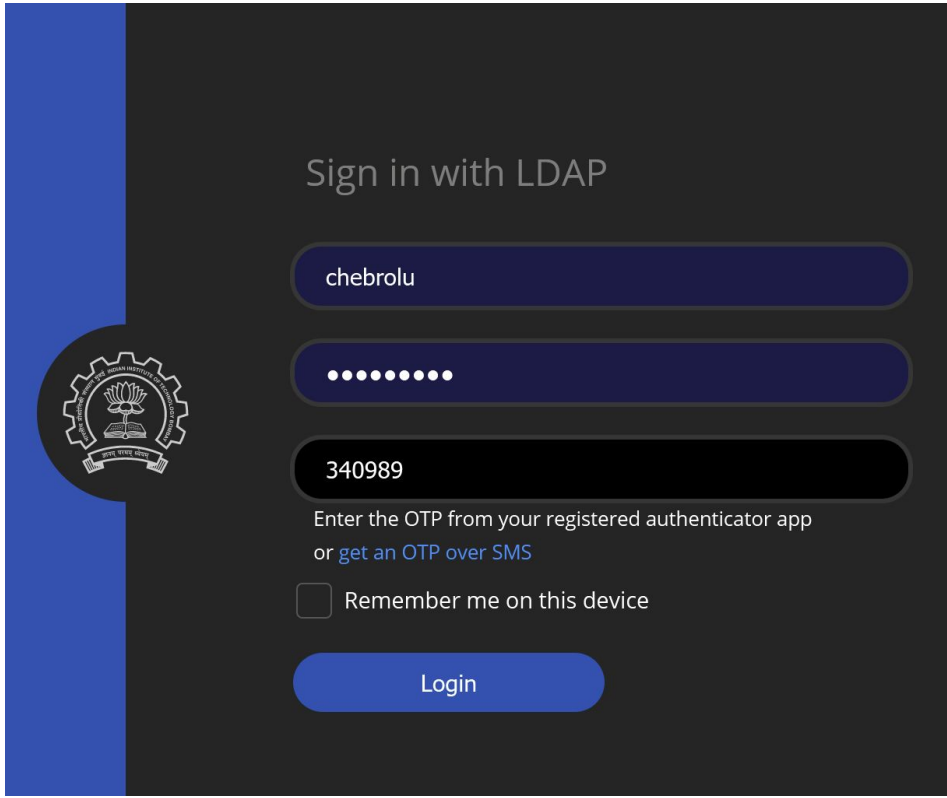
HTTP/2
Host: sso.iitb.ac.in

HTTP/2 200
….
(html/css that shows the given page)



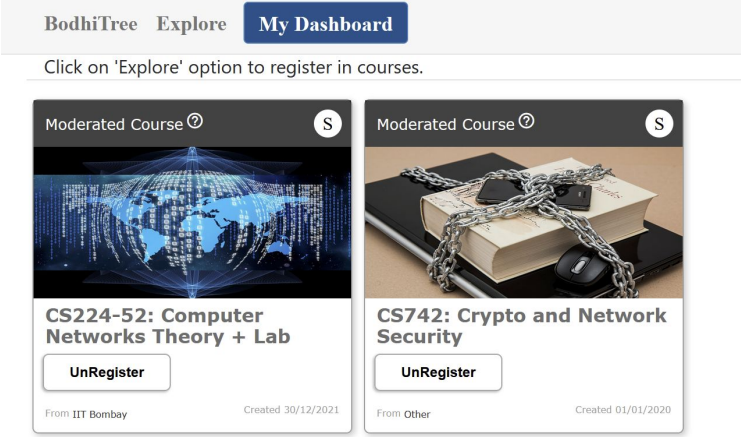
3.
POST /login HTTP/2
Host: sso.iitb.ac.in
…..

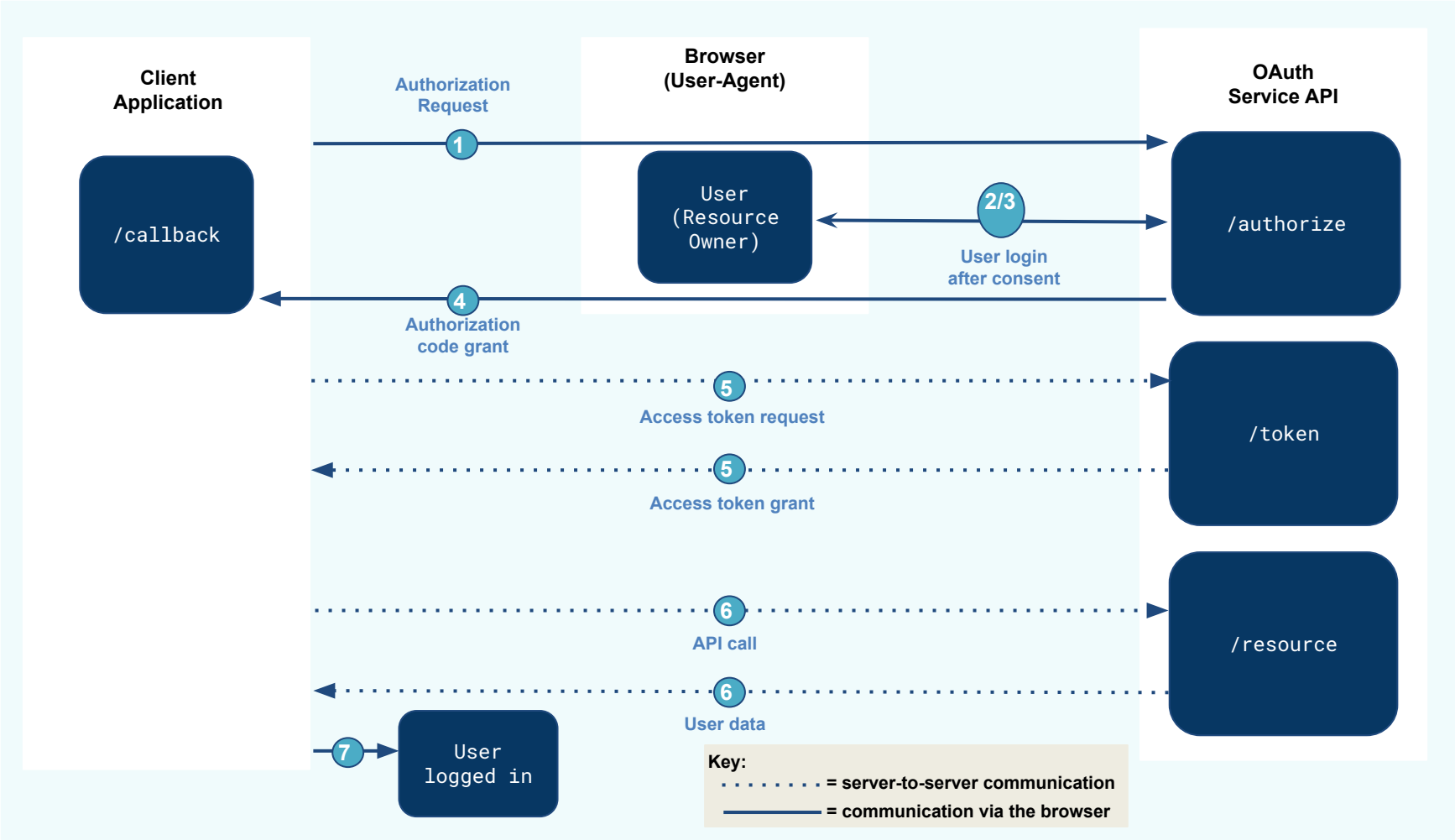
HTTP/2 302
….
location:
https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/?**code=9539bd3b66a6573ac1f589632d1611e5362adadd**&state=swueaxmguyfmqnzv



4. GET /accounts/login/?**code=9539bd3b66a6573ac1f589632d1611e5362adadd**&state=swueaxmguyfmqnzv HTTP/2
Host: flamingo.bodhi.cse.iitb.ac.in

HTTP/2 200
….




Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

Behind Scenes

5. Flamingo Server → sso.iitb.ac.in

POST /authorize HTTP/2
Host: sso.iitb.ac.in

client_id= bodhitreecseiitb
&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/
&grant_type=authorization_code&**code=9539bd3b66a6573ac1f589632d1611e5362adadd**

Response

{ "access_token": "kn912KLdgt…", "token_type": "Bearer", "expires_in": 3600, "scope": "openid profile", "refresh_token": "eyJhbGciOiJIU..”, ….}

6. Flamingo Server → resource.iitb.ac.in

GET /resource HTTP/2
Host: resource.iitb.ac.in
Authorization: Bearer kn912KLdgt…

JSON Response:
{ "username":"Kameswari Chebrolu",
"email":"kameswari@iitb.ac.in", … }

Summary of Authorization Code Grant

1. GET /accounts/sso/ HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

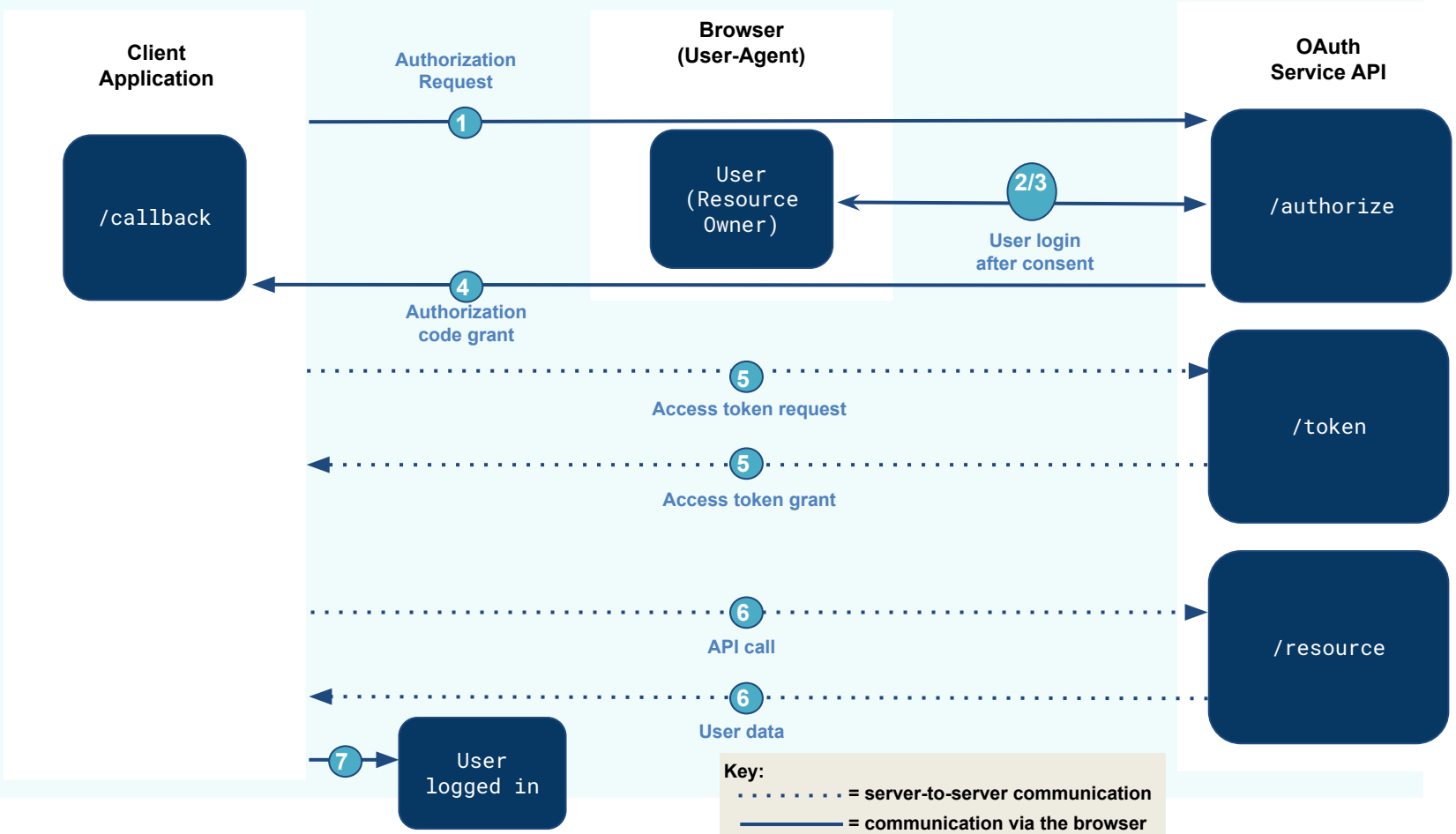
2. GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2
   Host: sso.iitb.ac.in

3. POST /login HTTP/2
   Host: sso.iitb.ac.in

4. GET /accounts/login/?code=9539bd3b66a6573ac1f589632d1611e5362adadd&state=swueaxmguyfmqnzv HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

5. POST /authorize HTTP/2 (from flamingo to sso)
   Host: sso.iitb.ac.in
   Body: client_id=bodhitreecseiitb&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&grant_type=authorization_code&code=9539bd3b66a6573ac1f589632d1611e5362adadd

6. GET /resource HTTP/2 (from flamingo to resource server)
   Host: resource.iitb.ac.in
   Authorization: Bearer kn912KLdgt…

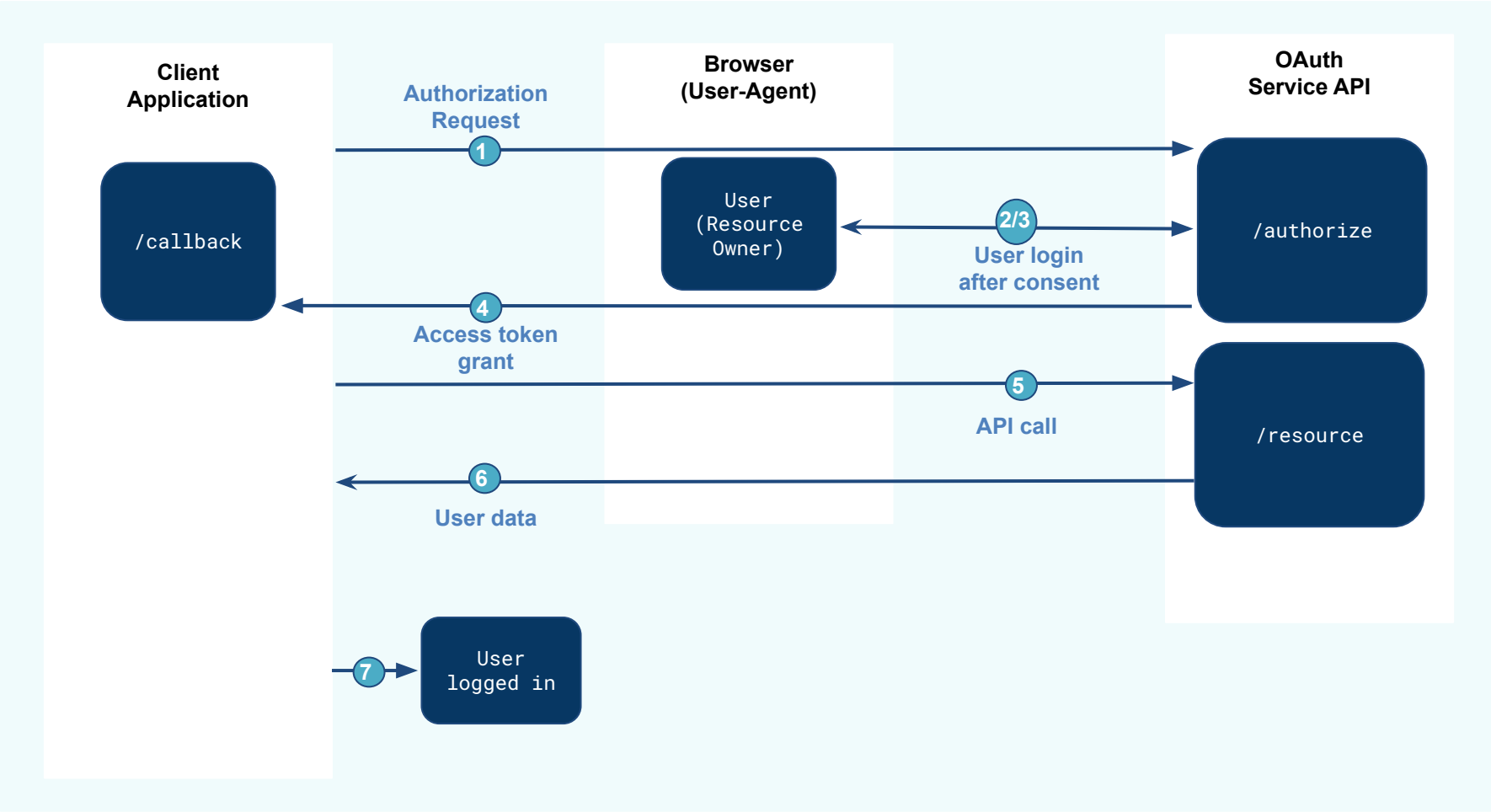

Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

Outline

- ~~OAuth background and terminology~~
- ~~Authorization Code Grant~~
- **Implicit Grant**
- OpenID Connect and SSO
- OAuth Vulnerabilities
- Defense Mechanisms
- Real Life Examples

Implicit Grant

- Much simpler
  - No behind the scenes part
    - No authorization code followed by access token
    - Client application receives access token immediately after the user gives consent


Client Application
Authorization Request
Browser (User-Agent)
OAuth Service API
1
/callback
User (Resource Owner)
2/3
/authorize
User login after consent
4
Access token grant
5
API call
/resource
6
User data
7
User logged in

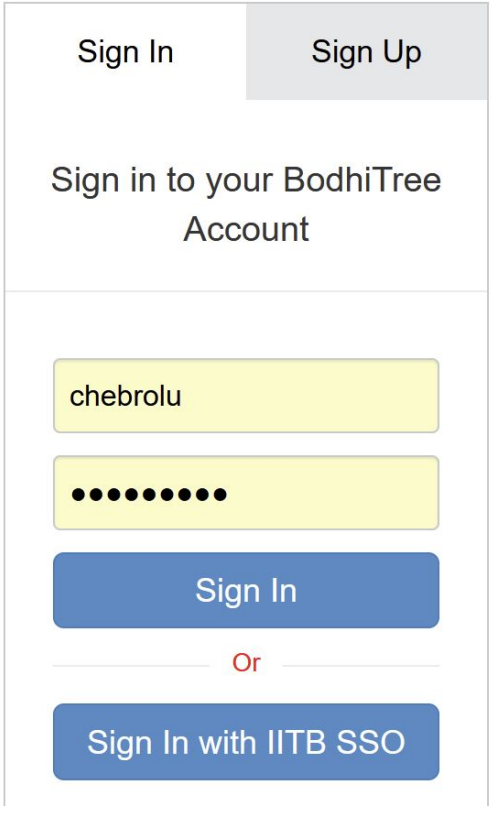
Implementations/Message-Sequence will differ in practice; but below captures the spirit!

1. User clicks on IITB SSO

GET /accounts/sso/ HTTP/2
Host: flamingo.bodhi.cse.iitb.ac.in

HTTP/2 302
…..

location=https://sso.iitb.ac.in/authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv

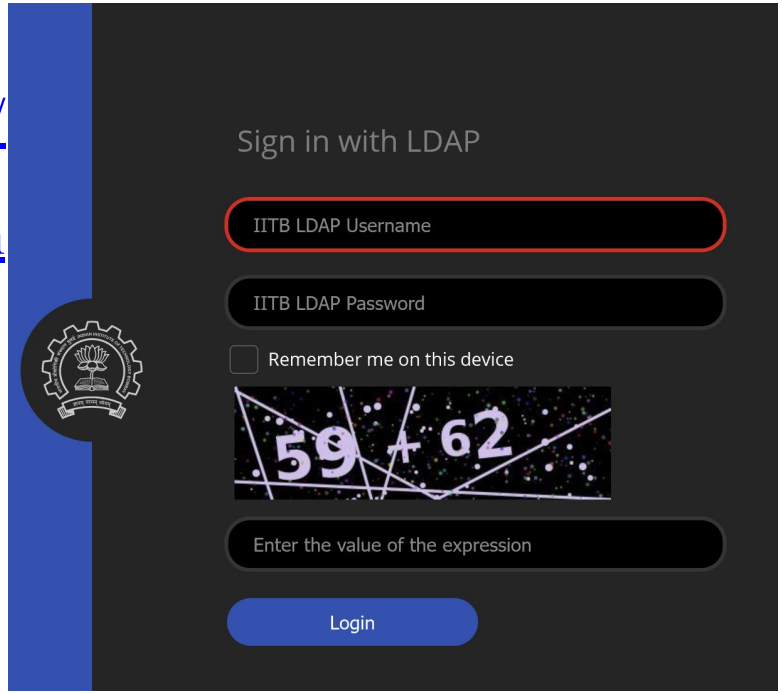


Authorization Grant Step 2 – FOR REFRENCE

GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv

HTTP/2
Host: sso.iitb.ac.in

HTTP/2 200
….
(html/css that shows the given page)



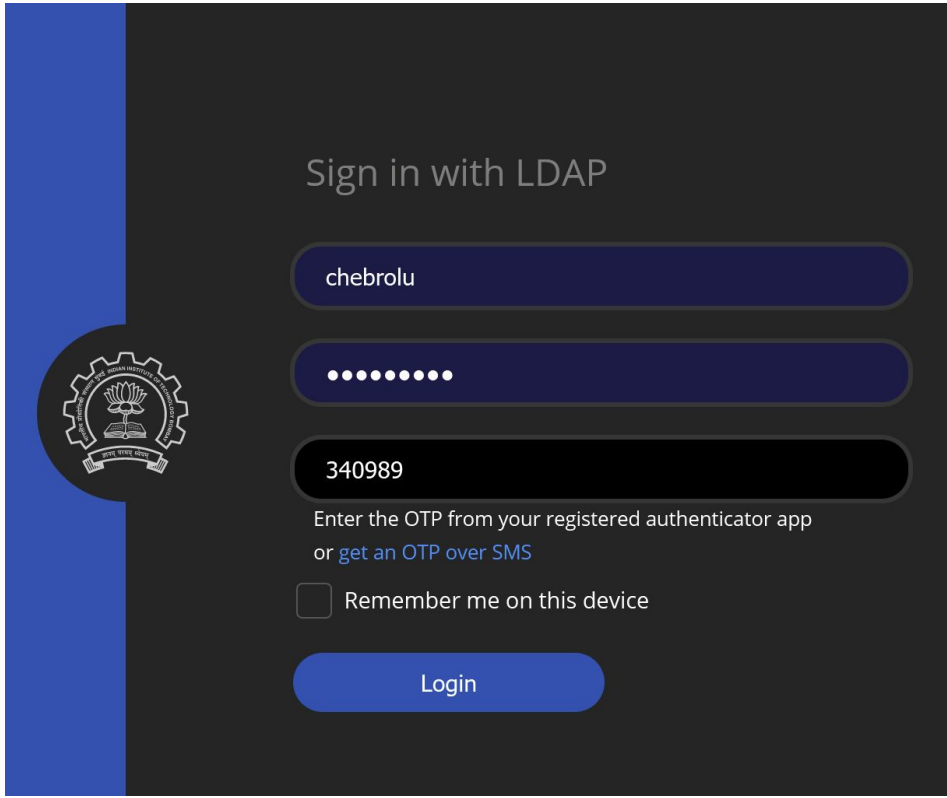
Implicit Grant step 2
(response type is different: token as opposed to code)

GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&**response_type=token**&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2

Authorization Grant Step 3: FOR REFERENCE

POST /login HTTP/2
Host: sso.iitb.ac.in
…..

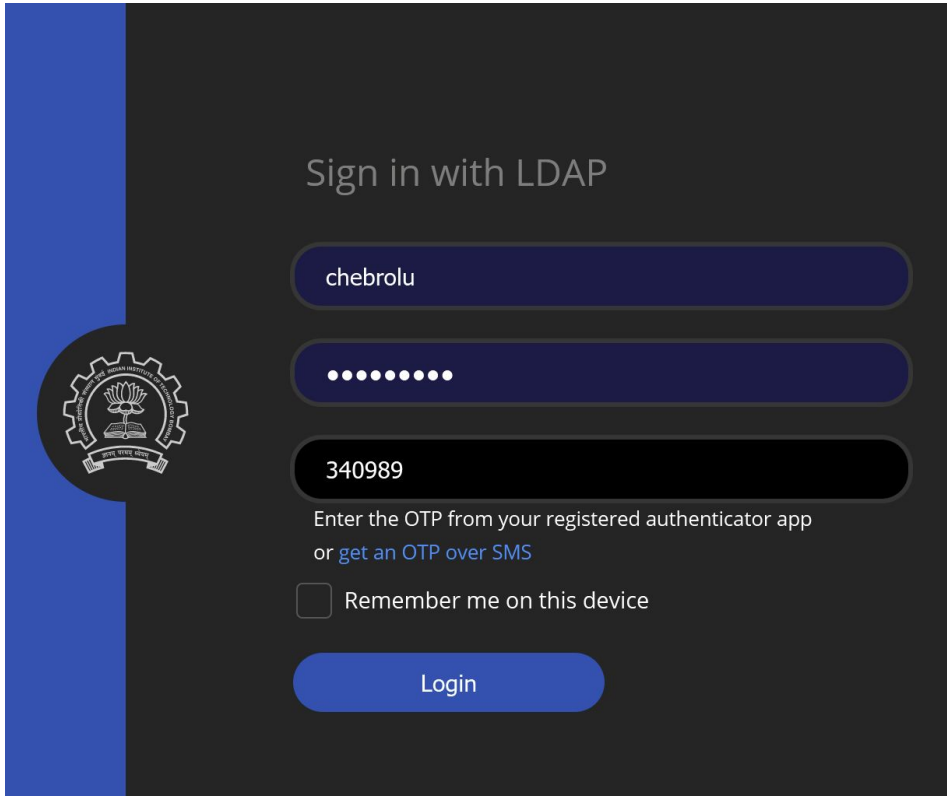
HTTP/2 302
….
location: https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/?**code=9539bd3b66a6573ac1f589632d1611e5362adadd**&state=swueaxmguyfmqnzv



Implicit Grant Step 3:

POST /login HTTP/2
Host: sso.iitb.ac.in
…..

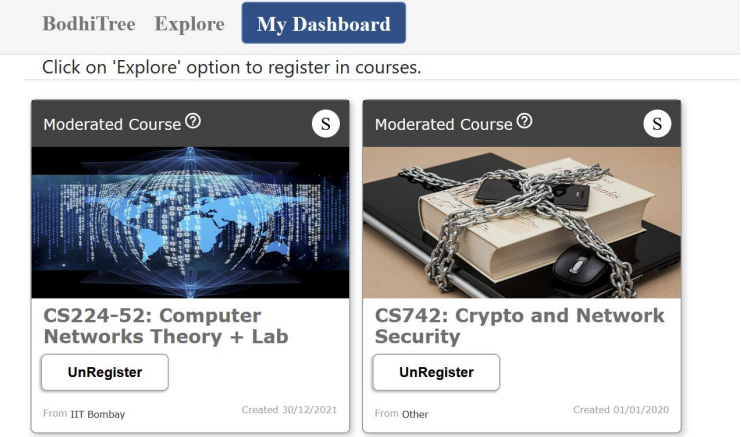
HTTP/2 302
….
location:
https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/**#access_token=9539bd3b66a6573ac1f589632d1611e5362adadd&token_type=Bearer&expires_in=5000&scope=openid%20profile**&state=swueaxmguyfmqnzv



Authorization grant Step 4 – FOR REFERENCE

GET /accounts/login/?**code=9539bd3b66a6573ac1f589632d1611e5362adadd**&state=swueaxmguyfmqnzv HTTP/2
Host: flamingo.bodhi.cse.iitb.ac.in

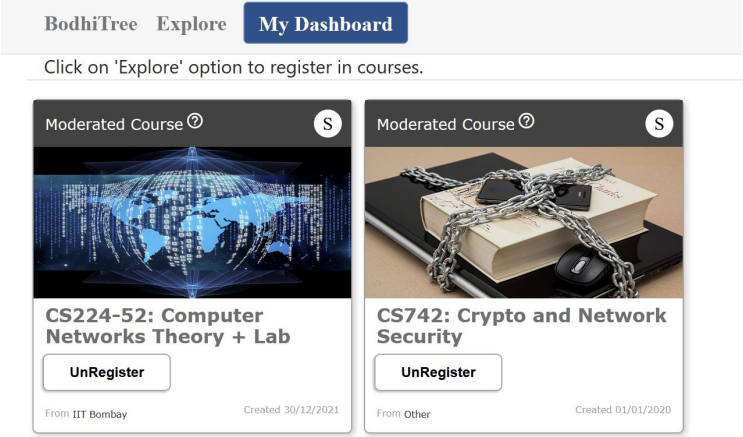
HTTP/2 200
….



Implicit grant Step 4
Behind Scenes becomes the following
User Browser will call back client application (Note: URL fragment not sent)

GET
/accounts/login/
Host: flamingo.bodhi.cse.iitb.ac.in
HTTP/2

HTTP/2 200
(script)



```
<script>
const urlSearchParams = new URLSearchParams(window.location.hash.substr(1));
const token = urlSearchParams.get('access_token');
fetch('https://resource.iitb.ac.in/resource', {
        method: 'GET',
        headers: {
        'Authorization': 'Bearer ' + token,
        'Content-Type': 'application/json'
        }
})
.then(r => r.json())
.then(j =>
        fetch('/authenticate', {
        method: 'POST',
        headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json'
        },
        body: JSON.stringify({
        email: j.email,
        username: j.sub,
        })
        }).then(r => document.location = '/'))
</script>
```

Parses the URL fragment to get access token

Contact the resource server, pass the token (HTTP GET request)

The resulting JSON response is parsed

Then contact flamingo.bodhi.cse.iitb.ac.in/authenticate end point via POST and passes on obtained username/email etc

Post this visit / endpoint

- Suited to single-page applications which cannot easily store the client_secret on the back-end!

Summary of Implicit Grant

1. GET /accounts/sso/ HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

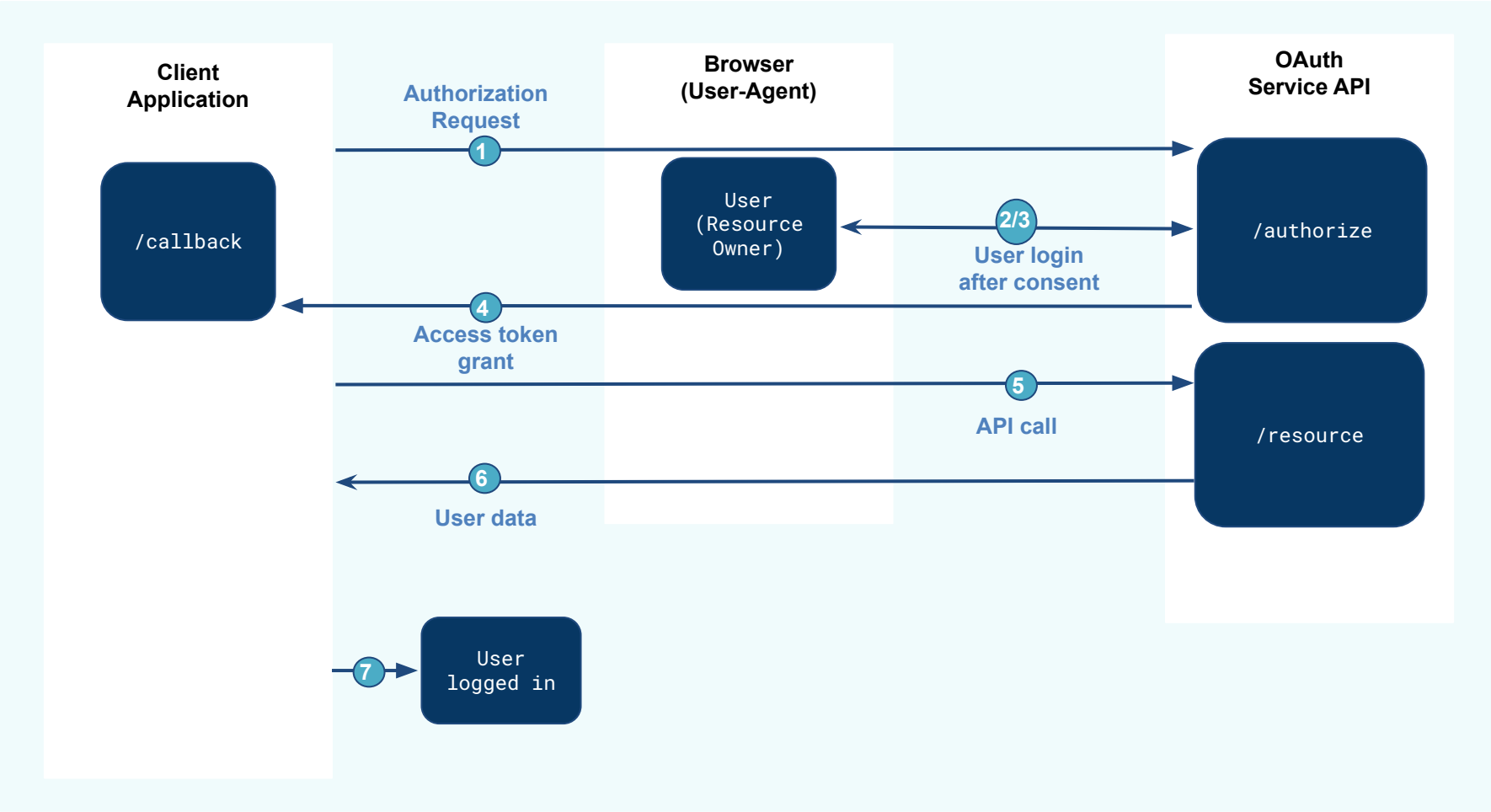
2. GET /authorization?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=token&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2
   Host: sso.iitb.ac.in

3. POST /login HTTP/2
   Host: sso.iitb.ac.in

4. GET /accounts/login HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

5. GET /resource HTTP/2 (from browser to resource server)
   Host: resource.iitb.ac.in
   Authorization: Bearer 9539….

6. POST /authenticate HTTP/2 (from browser to client-app)
   Host: flamingo.bodhi.cse.iitb.ac.in
   JSON Response: { "username":"Kameswari Chebrolu", "email":"kameswari@iitb.ac.in", … }


Client Application
Authorization Request
Browser (User-Agent)
OAuth Service API
1
/callback
User (Resource Owner)
2/3
/authorize
User login after consent
4
Access token grant
5
API call
/resource
6
User data
7
User logged in

Refresh Tokens

- Long-lived sessions or applications requiring continuous access to user data
- Asking user for permission often may be inconvenient
- Refresh tokens can be used to obtain a new access token without requiring the user to log in again
  - Have a longer lifespan than the access token
  - Can be revoked also as needed
- Note refresh tokens are not used in implicit flow grant

Behind Scenes (Initial Authorization)

5. Flamingo Server → sso.iitb.ac.in

POST /authorize HTTP/2
Host: sso.iitb.ac.in

client_id=12345&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/
&grant_type=authorization_code&**code=9539bd3b66a6573ac1f589632d1611e5362adadd**

Response

{ "access_token": "kn912KLdgt…", "token_type": "Bearer", "expires_in": 3600, "scope": "openid profile", "**refresh_token": "eyJhbGciOiJIU..**", ….}

- After the initial access token expires, client-app uses the refresh token to request a new one
- Client-app can now use the new access token to make authenticated API requests to the resource server
- Refresh tokens are often long-lived, so they need to be stored securely
- Authorization server may revoke refresh tokens under certain conditions (e.g., user revokes consent, session termination)

```
POST /authorize HTTP/1.1
Host: sso.iitb.ac.in
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&refresh_token=eyJhbGciOiJIU…
&client_id=12345
&client_secret=SECRET
```

```
{
  "access_token": "NEW_ACCESS_TOKEN",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "NEW_REFRESH_TOKEN"
}
```


refresh_token above is optional

Outline

- ~~OAuth background and terminology~~
- ~~Authorization Code Grant~~
- ~~Implicit Grant~~
- **OpenID Connect and SSO**
- OAuth Vulnerabilities
- Defense Mechanisms
- Real Life Examples

Open ID Connect

- OAuth was initially designed for authorization, not authentication
- But many websites started using it for authentication
  - Request read access to some basic user data and if granted assume user authenticated by OAuth provider
- Different client applications implemented it differently
  - Also, each client application had different implementation for each provider!

- OpenID Connect extends OAuth protocol to provide dedicated identity and authentication
  - Adds standardized, identity-related features
  - Scopes are standardized and same for all providers,
    - An extra response type: id_token is added

- ID token: a response type, a JSON web token (JWT) signed with a JSON web signature (JWS)
  - JWT payload contains a list of claims based on the scope that was initially requested
- Note: multiple response types are supported by OAuth
  - E.g. response_type=id_token code
    - Both an ID token and access token will be sent to the client application at the same time

- Terminology is different
  - Relying party → Client-app
  - End user → Resource Owner
  - OpenID provider → OAuth service provider that supports OpenID Connect.
- Claims: key:value pairs that represent information about the user
  - "last_name":"Chebrolu"
- Scopes: profile, email, address, phone etc
  - Each of these scopes corresponds to read access for a subset of claims

Summary of Authorization Code Grant

1. GET /accounts/sso/ HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

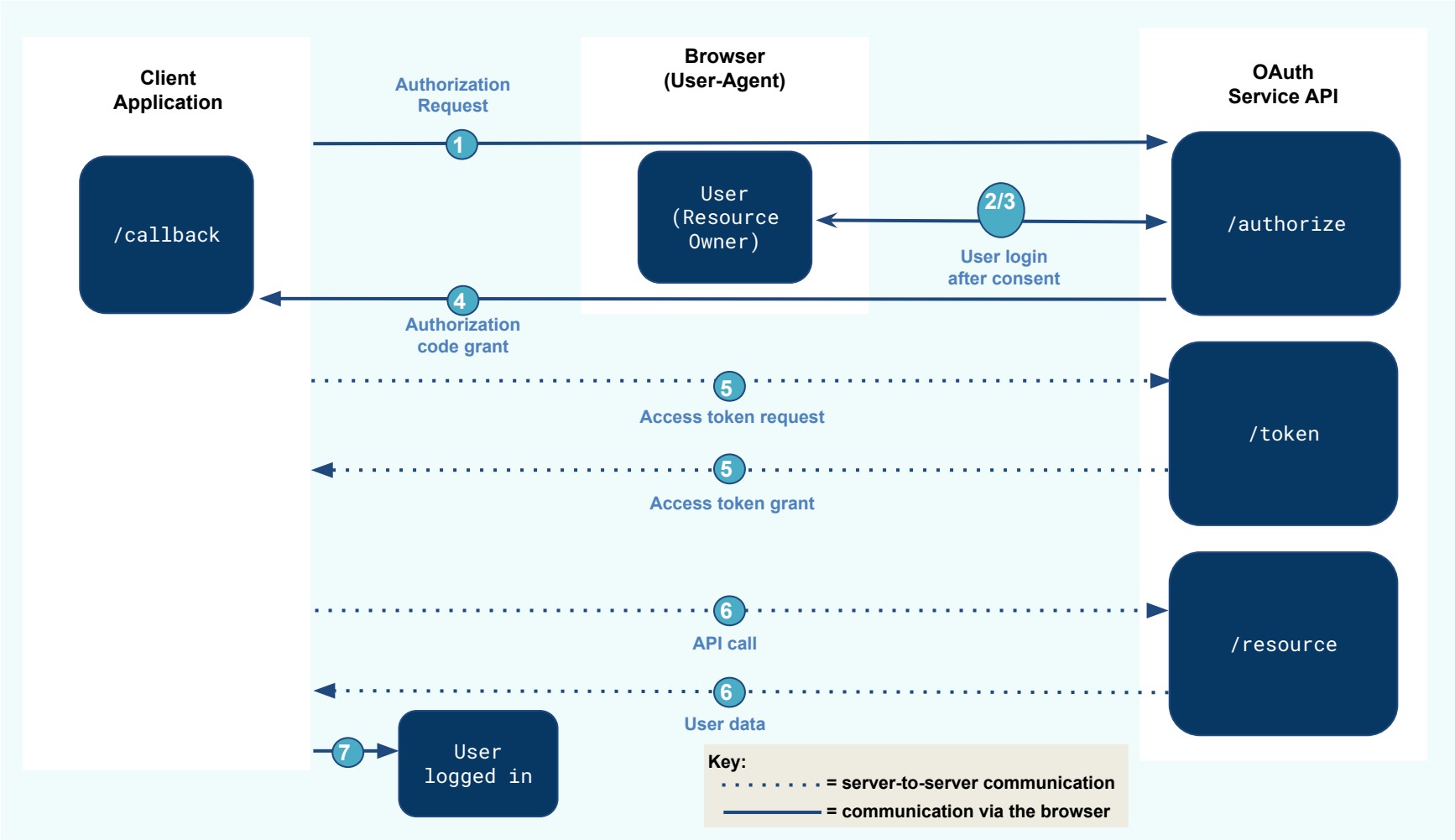
2. GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2
   Host: sso.iitb.ac.in

3. POST /login HTTP/2
   Host: sso.iitb.ac.in

4. GET /accounts/login/?code=9539bd3b66a6573ac1f589632d1611e5362adadd&state=swueaxmguyfmqnzv HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

5. POST /authorize HTTP/2 (from flamingo to sso)
   Host: sso.iitb.ac.in
   Body: client_id=bodhitreecseiitb&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&grant_type=authorization_code&code=9539bd3b66a6573ac1f589632d1611e5362adadd

6. GET /resource HTTP/2 (from flamingo to resource server)
   Host: resource.iitb.ac.in
   Authorization: Bearer kn912KLdgt…


Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

Behind Scenes

5. Flamingo Server → sso.iitb.ac.in

POST /authorization HTTP/2
Host: sso.iitb.ac.in

client_id=12345&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&grant_type=authorization_code&**code=9539bd3b66a6573ac1f589632d1611e5362adadd**

Response


```
{
    "access_token": "kn912KLdgt…",
    "token_type": "bearer",
    "expires_in": 3600,
    "refresh_token": "bKLsjIRKPO…",
    "scope": "openid profile email",
    "id_token": "JoSATYngd…"
}
```

Single Sign On (SSO)

- A mechanism where a user can authenticate once and access multiple applications or services
  - No need to log in again for each service
- Makes use of OpenID Connect (OIDC) built on top of OAuth 2.0
- SSO server acts as the OpenID Connect provider (OP)
  - User is authenticated centrally → As long as user session active, no need to login again
    - Handled via usual session token/cookies/JWTs etc
  - Just pass a new token for the other client-app

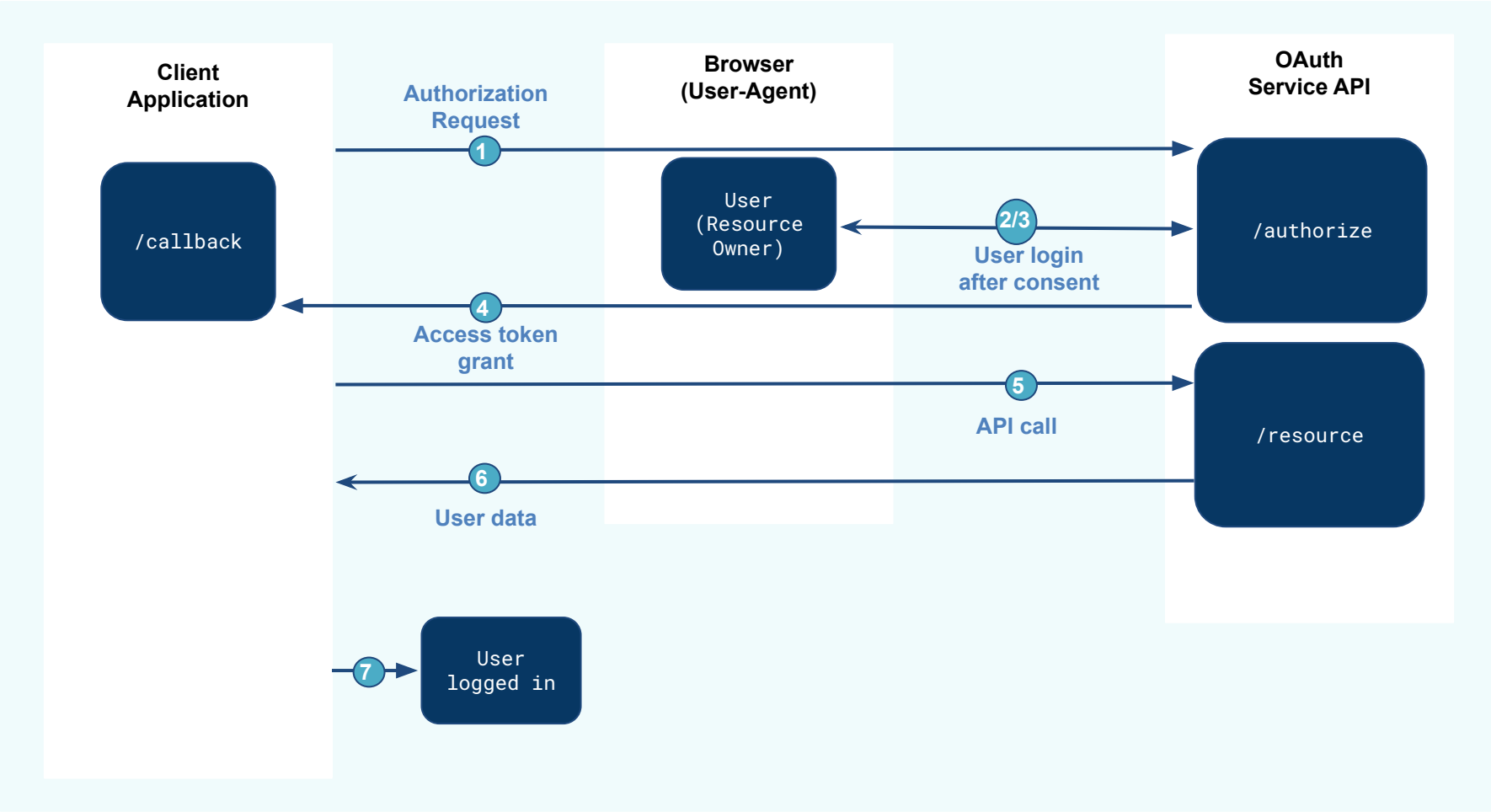
Explanation

Outline

- ~~OAuth background and terminology~~
- ~~Authorization Code Grant~~
- ~~Implicit Grant~~
- ~~OpenID Connect and SSO~~
- **OAuth Vulnerabilities**
- Defense Mechanisms
- Real Life Examples

Authentication bypass in Implicit grant

- See next slide
- Client-app has no way to check if credentials belong to the right person
- Attacker can establish a session, but ultimately pass on wrong user data
  - Can login to other's account!


Client Application
Authorization Request
Browser (User-Agent)
OAuth Service API
1
/callback
User (Resource Owner)
2/3
/authorize
User login after consent
4
Access token grant
5
API call
/resource
6
User data
7
User logged in

Summary of Implicit Grant

1. GET /accounts/sso/ HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

2. GET /authorization?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=token&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2
   Host: sso.iitb.ac.in

3. POST /login HTTP/2
   Host: sso.iitb.ac.in

4. GET /accounts/login HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

5. GET /resource HTTP/2 (from browser to resource server)
   Host: resource.iitb.ac.in
   Authorization: Bearer 9539….

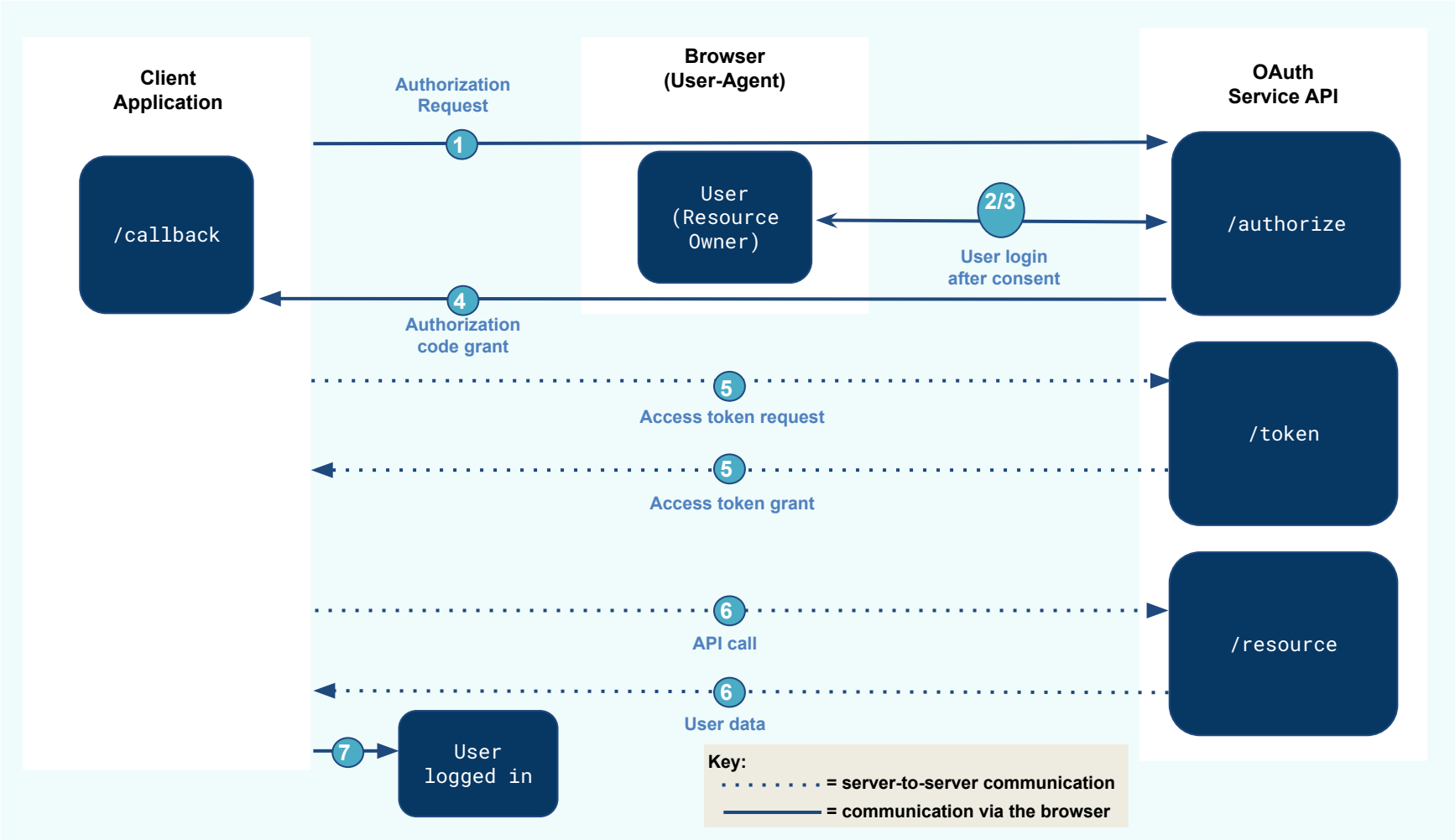
6. POST /authenticate HTTP/2 (from browser to client-app)
   Host: flamingo.bodhi.cse.iitb.ac.in
   JSON Response: { "username":"Kameswari Chebrolu", "email":"kameswari@iitb.ac.in", … }

Not using state parameter

- State parameter is optional
- Consider below example involving “Authorized Flow Grant”
  - Client App lets you link social media account to your account (so you can login via that also)
    - Two login methods: usual username/password as well as via social media account (oauth based)
    - Done via a link button which launches below request

GET
authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile HTTP/2
Host: sso.iitb.ac.in

(Note, user directly contacts the oauth provider, client app does not pass on any state parameter)


Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

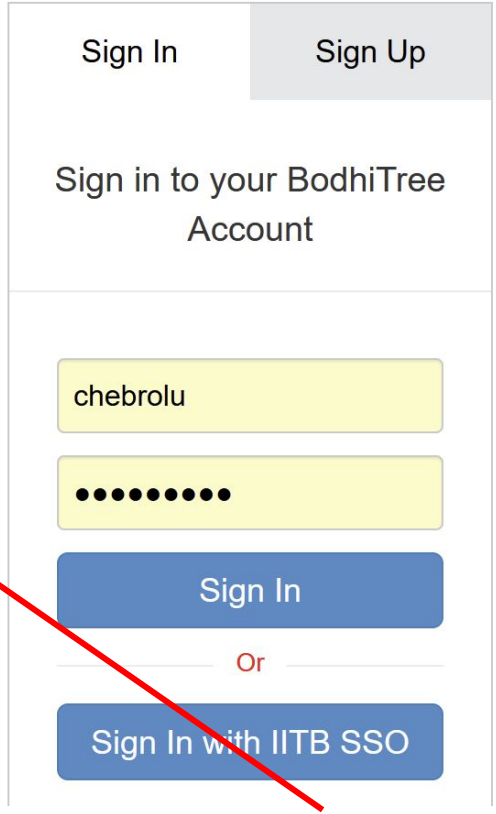
Implementations/Message-Sequence will differ in practice; but below captures the spirit!

1. User clicks on IITB SSO

GET /accounts/sso/ HTTP/2
Host: flamingo.bodhi.cse.iitb.ac.in

HTTP/2 302
…..

location=https://sso.iitb.ac.in/authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv



- Goal: Attacker wants to link her social media account to some admin account on the client-app
  - If successful, can log into admin account using her social media credentials
  - Need to somehow trick admin to link her social media account

Summary of Authorization Code Grant

1. GET /accounts/sso/ HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

2. GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2
   Host: sso.iitb.ac.in

3. POST /login HTTP/2
   Host: sso.iitb.ac.in

4. GET /accounts/login/?code=9539bd3b66a6573ac1f589632d1611e5362adadd HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

5. POST /authorize HTTP/2 (from flamingo to sso)
   Host: sso.iitb.ac.in
   Body: client_id=bodhitreecseiitb&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&grant_type=authorization_code&code=9539bd3b66a6573ac1f589632d1611e5362adadd

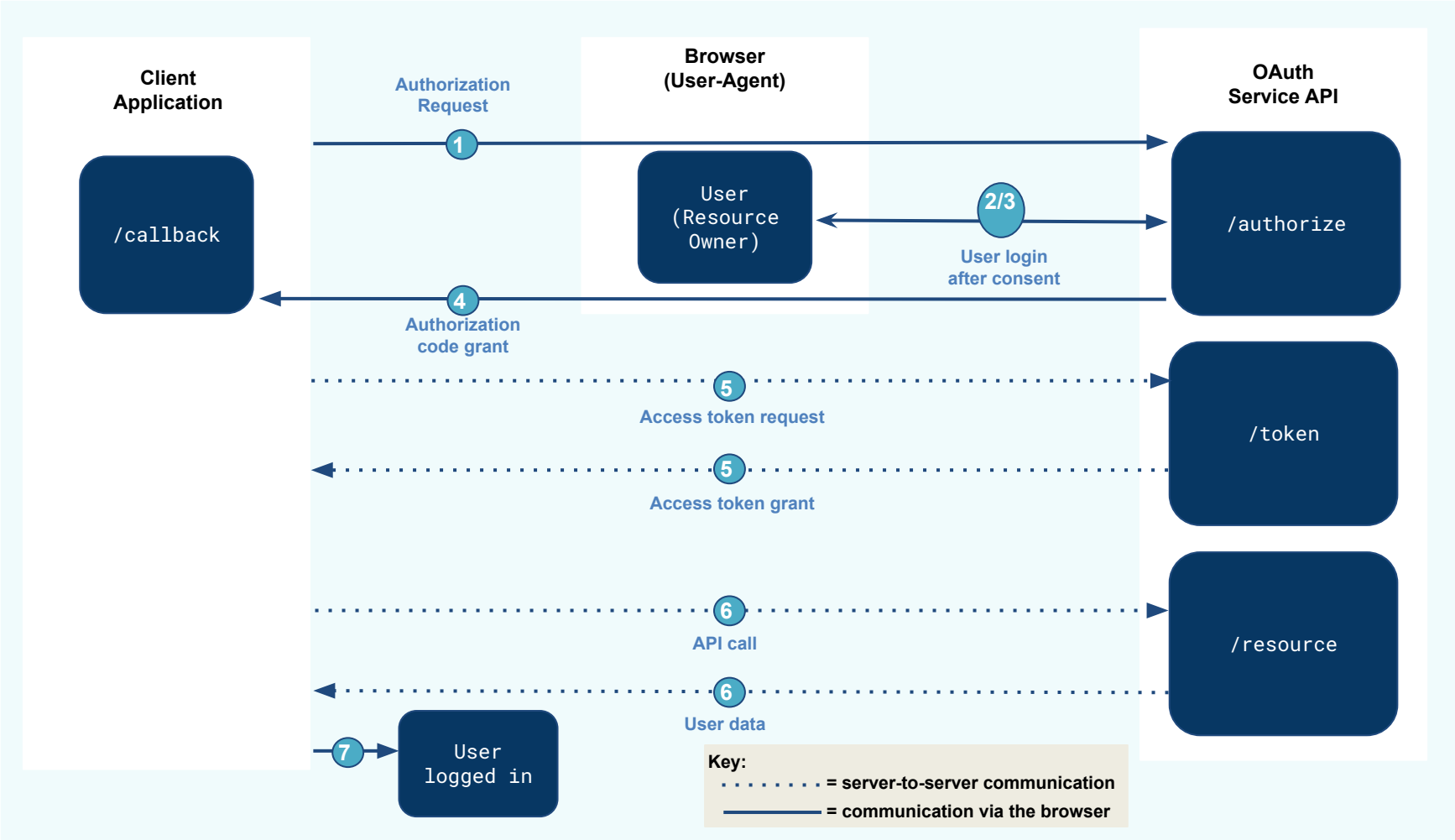
6. GET /resource HTTP/2 (from flamingo to resource server)
   Host: resource.iitb.ac.in
   Authorization: Bearer kn912KLdgt…

Attacker Steps

- Setup a malicious website or send email, but somehow makes admin click on a malicious link
  - CSRF style attack

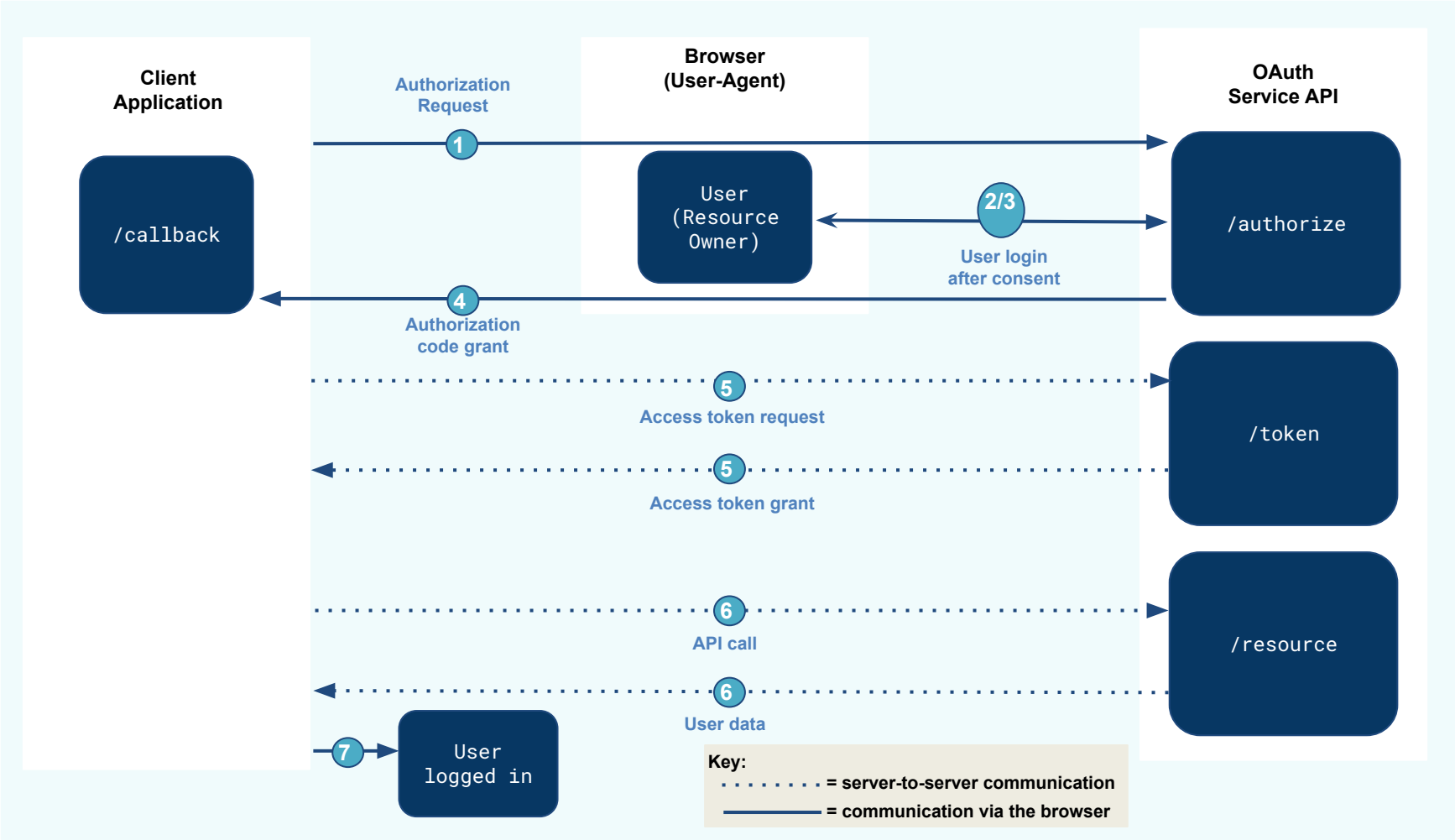
```
<iframe
src="https://client-app.com/callback?code=Attacker’s
code"></iframe>
```

- Attacker’s code obtained by attacker by trying to link his own social media account
- Since state is not tracked, one session’s “code” is being used in another


Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

- Assume admin is currently logged in the client-app website
  - Logged in using regular username:password
  - Some session cookie is installed in admin's browser for this client-app
- Admin then clicked the malicious link and sends a request to client-app with attacker's code (step 4)
  - This will include admin's session cookie to that website
  - Client-app believes it is the admin user since session cookie is correct!
  - Client app will do relevant behind the scene steps and finally link attacker's social media account to admin

- If State parameter was used
  - First step is needed to get the state parameter from client-app
  - In step 4, the same state parameter has to be passed back to client-app
  - Attacker can get this state parameter for his session, along with code but he cannot predict the state parameter of a session corresponding to admin
    - Client-app would expect the same state parameter it passed to admin in step 1 to reflect back in step 4


Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

Leaking authorization code/tokens

- Can happen if OAuth service fails to validate redirect URI properly
- CSRF style attack: Trick victim's browser into initiating an OAuth flow
  - But send the code or token to an attacker-controlled redirect_uri

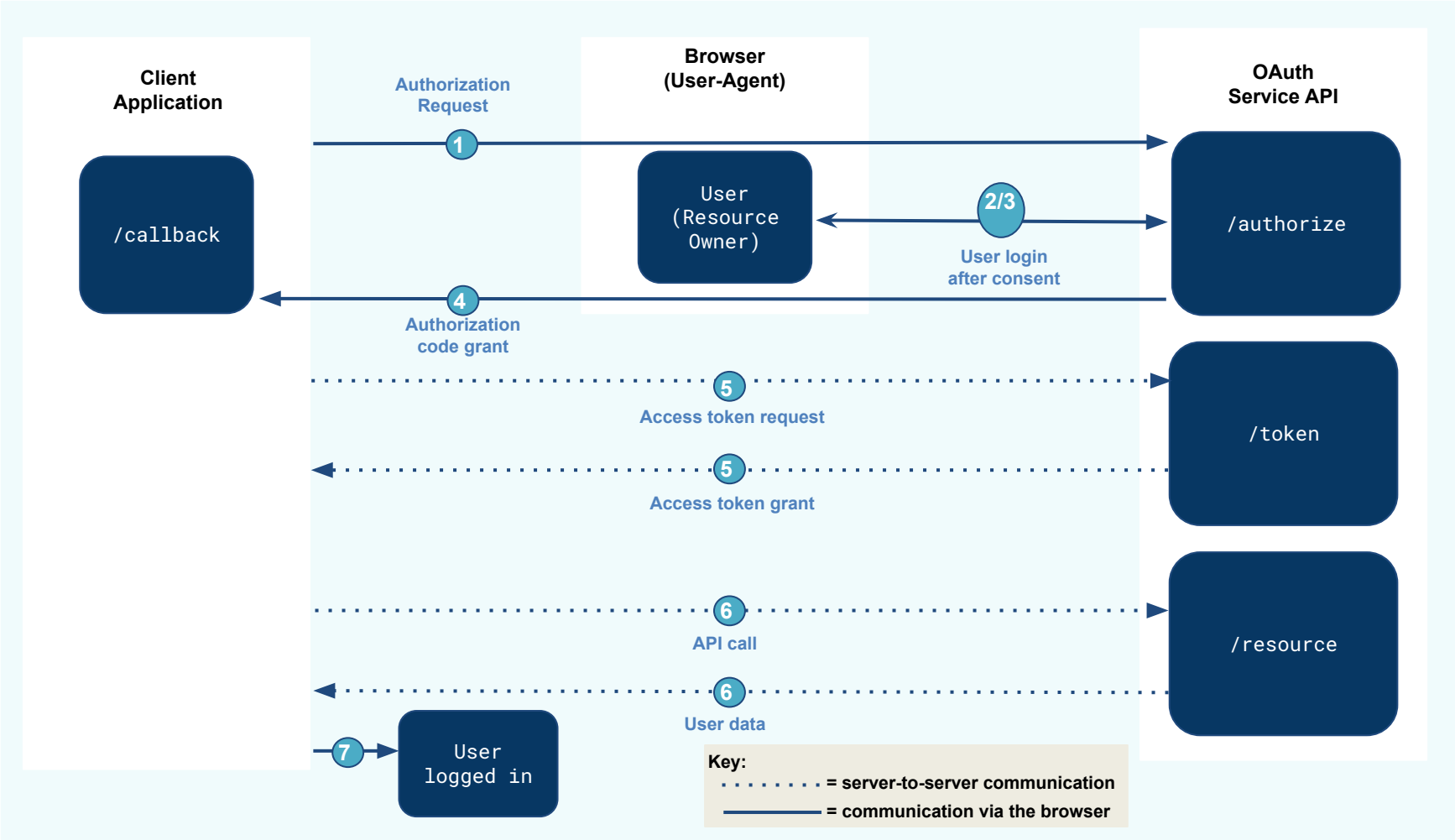
Attacker created a malicious website with below iframe and trick victim to access the page

- <iframe src="https://oauth-server.com/authorize?client_id=12345&redirect_uri=https://attacker.com/callback&response_type=code&scope=openid%20profile%20email"></iframe>
  - Essentially Step 2; initiated by victim but via attacker website
  - Client id is public info as such!
  - Can prompt victim to enter login credentials -> victim can get suspicious
    - However if it is SSO and already authenticated, token passed via 302 redirect (user unaware)
  - State Parameter is not needed here (optional in standard)
    - Has no relevance to Oauth provider
    - It just reflects it back to help the client-app

GET
/authorize?client_id=12345&redirect_uri=https://attacker.com/callback&response_type=code&scope=openid%20profile%20email

…..

HTTP/2 302
….
 location:
https://attacker.com/callback/?code=9539bd3b66a6573ac1f589632d1611e5362adadd


Client Application
/callback
Authorization Request
1
Browser (User-Agent)
User (Resource Owner)
2/3
User login after consent
OAuth Service API
/authorize
4
Authorization code grant
5
Access token request
/token
5
Access token grant
6
API call
/resource
6
User data
7
User logged in
Key:
= server-to-server communication
= communication via the browser

Attacker:

- As part of steps 2/3/4 attacker will get a code
  - Note step 4 message is sent to attacker, not client-app
  - Code belongs to Victim (Victim was tricked to do these steps)
- Attacker can start a session with client-app (step1) and pass code obtained above in his own step 4 to client-app
  - If client-app is using state parameter, just include it from step1 into step 4
  - Use of state parameter cannot defend against this attack

- More secure authorization servers will require a redirect_uri parameter sent in step 2 to be checked with one included in step 5
  - Only if they match, respond

Summary of Authorization Code Grant

1. GET /accounts/sso/ HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

2. GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2
   Host: sso.iitb.ac.in

3. POST /login HTTP/2
   Host: sso.iitb.ac.in

4. GET /accounts/login/?code=9539bd3b66a6573ac1f589632d1611e5362adadd&state=swueaxmguyfmqnzv HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

5. POST /authorize HTTP/2 (from flamingo to sso)
   Host: sso.iitb.ac.in
   Body:
client_id=bodhitreecseiitb&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&grant_type=authorization_code&code=9539bd3b66a6573ac1f589632d1611e5362adadd

6. GET /resource HTTP/2 (from flamingo to resource server)
   Host: resource.iitb.ac.in
   Authorization: Bearer kn912KLdgt…

Flawed scope validation

- Token/Code should allow client application to access only the scope that was approved by the user
- But attacker can upgrade the access token due to flawed validation by the OAuth service

- Scope upgrade: authorization code flow
  - Difficult in practice since this code grant happens behind the scenes
  - But attacker can register their own malicious client application with the OAuth service
    - Attacker's malicious client application initially requests access to the user's email address
    - After user approves request, malicious client application receives an authorization code
    - App then adds another scope parameter to the code/token exchange request

Summary of Authorization Code Grant

1. GET /accounts/sso/ HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

2. GET /authorize?client_id=bodhitreecseiitb&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&response_type=code&scope=openid%20profile&state=swueaxmguyfmqnzv HTTP/2
   Host: sso.iitb.ac.in

3. POST /login HTTP/2
   Host: sso.iitb.ac.in

4. GET /accounts/login/?code=9539bd3b66a6573ac1f589632d1611e5362adadd&state=swueaxmguyfmqnzv HTTP/2
   Host: flamingo.bodhi.cse.iitb.ac.in

5. POST /authorize HTTP/2 (from flamingo to sso)
   Host: sso.iitb.ac.in
   Body: client_id=bodhitreecseiitb&client_secret=SECRET&redirect_uri=https://flamingo.bodhi.cse.iitb.ac.in/accounts/login/&grant_type=authorization_code&code=9539bd3b66a6573ac1f589632d1611e5362adadd&scope=**something else**

6. GET /resource HTTP/2 (from flamingo to resource server)
   Host: resource.iitb.ac.in
   Authorization: Bearer kn912KLdgt…

- Defense: Before granting resource access, server has to validate the scope within this request (step 5) with the initial authorization request (step2)
  - Else can share more info than authorized!

Outline

- ~~OAuth background and terminology~~
- ~~Authorization Code Grant~~
- ~~Implicit Grant~~
- ~~OpenID Connect and SSO~~
- ~~OAuth Vulnerabilities~~
- **Defense Mechanisms**
- **Real Life Examples**

Defense: OAuth Service Providers

- Protocol is well defined, Good Implementation is the defense
  - Look at OAuth2.1 that covers best security practices
- Ensure client applications register with service provider a whitelist of valid redirect_uris
  - Use strict byte-for-byte comparison to validate the URI in any incoming requests (including later requests)

- Enforce use of the state parameter (unguessable)
- On resource server, verify access token was issued to the same client_id that is making the request
  - Also check scope being requested matches the scope for which the token was originally granted
- Use encryption and integrity checks to protect token/codes values both in transit and at rest

Defense: OAuth Client Applications

- Fully understand how OAuth works, vulnerabilities arise due to lack of understanding
- Use state parameter even though it is optional
- If using OpenID Connect id_token, validate the JW token
- Be careful with authorization codes - they may be leaked via Referer headers
- Monitor and analyze access patterns to detect and mitigate suspicious activities
  - E.g. Unusually high request rates or unauthorized access attempts

Real Life Examples

- Slack OAuth2 "redirect_uri" Bypass to steal access tokens: https://hackerone.com/reports/2575/
- Swiping Facebook Official Access Tokens: http://philippeharewood.com/swiping-facebook-official-access-tokens/

References

- https://portswigger.net/web-security/oauth